

Introduction to Computer Architecture

Chapter 5

Large and Fast: Exploiting Memory Hierarchy

Hyungmin Cho

Department of Computer Science and Engineering
Sungkyunkwan University

Memory Technology

■ Access latency

❖ Static RAM (SRAM)

- 0.5ns – 2.5ns (16KB~64KB)
- 5ns – 10ns (1MB~4MB)

❖ Dynamic RAM (DRAM)

- 50ns – 70ns (GBs)

❖ Flash storage

- 5 – 50μs (TBs)

❖ Magnetic disk

- 5ms – 20ms (10+ TBs)

embedded in CPU
L cache

1 GHz CPU SRAM 1~? cycles
5 cycles

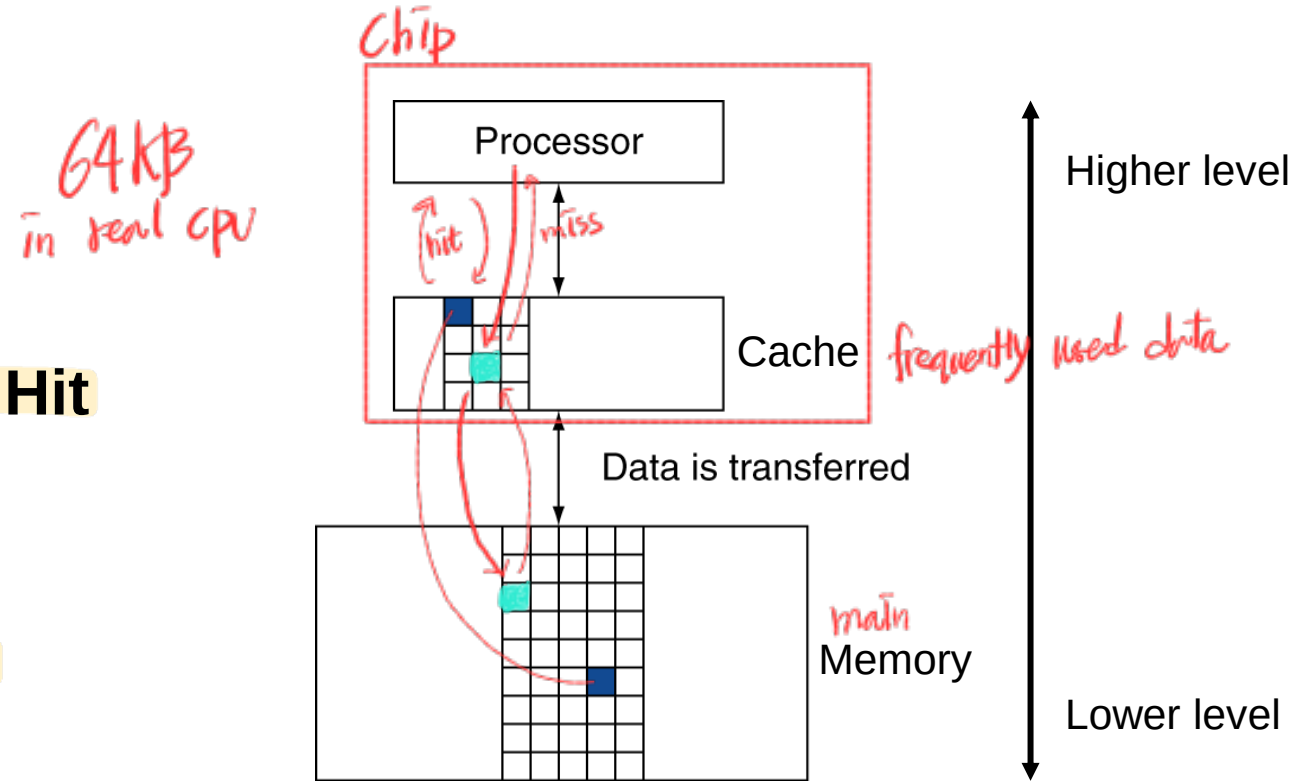
100 cycles

main memory

swapping

Memory Hierarchy Levels

- **Cache:** small, fast memory near the core
- **Block (a.k.a. line):** unit of transfer
 - ❖ May be multiple words
- If accessed data is present in cache: **Hit**
 - ❖ Hit ratio: $\#hits / \#accesses$
- If accessed data is not present: **Miss**
 - ❖ Miss: block copied from lower level (fill)
 - Time taken: miss penalty
 - Miss ratio: $\#misses / \#accesses = 1 - \text{hit ratio}$



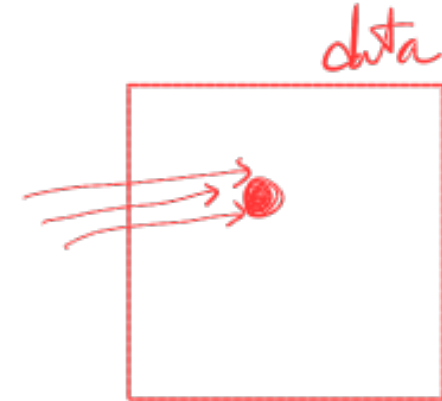
Principle of Locality

- Programs access only a small portion of the memory at a given time

- **Temporal locality**

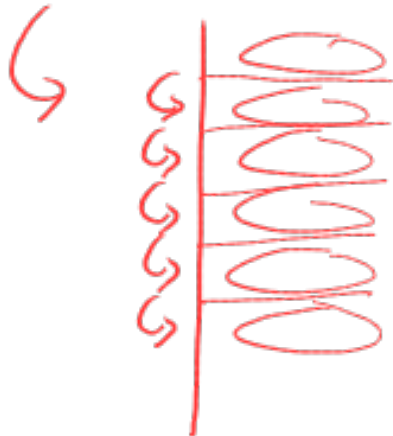
- ❖ Items accessed recently are likely to be accessed again soon
- ❖ e.g., instructions in a loop, index variables

array[i] in while access

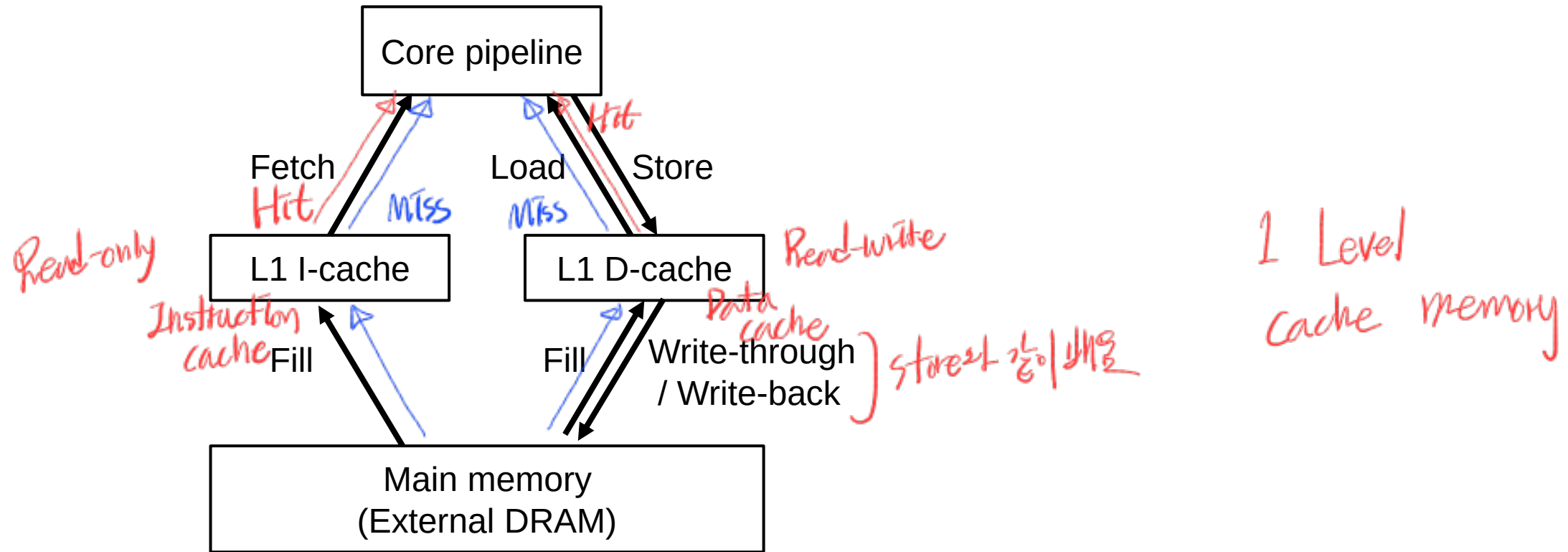


- **Spatial locality**

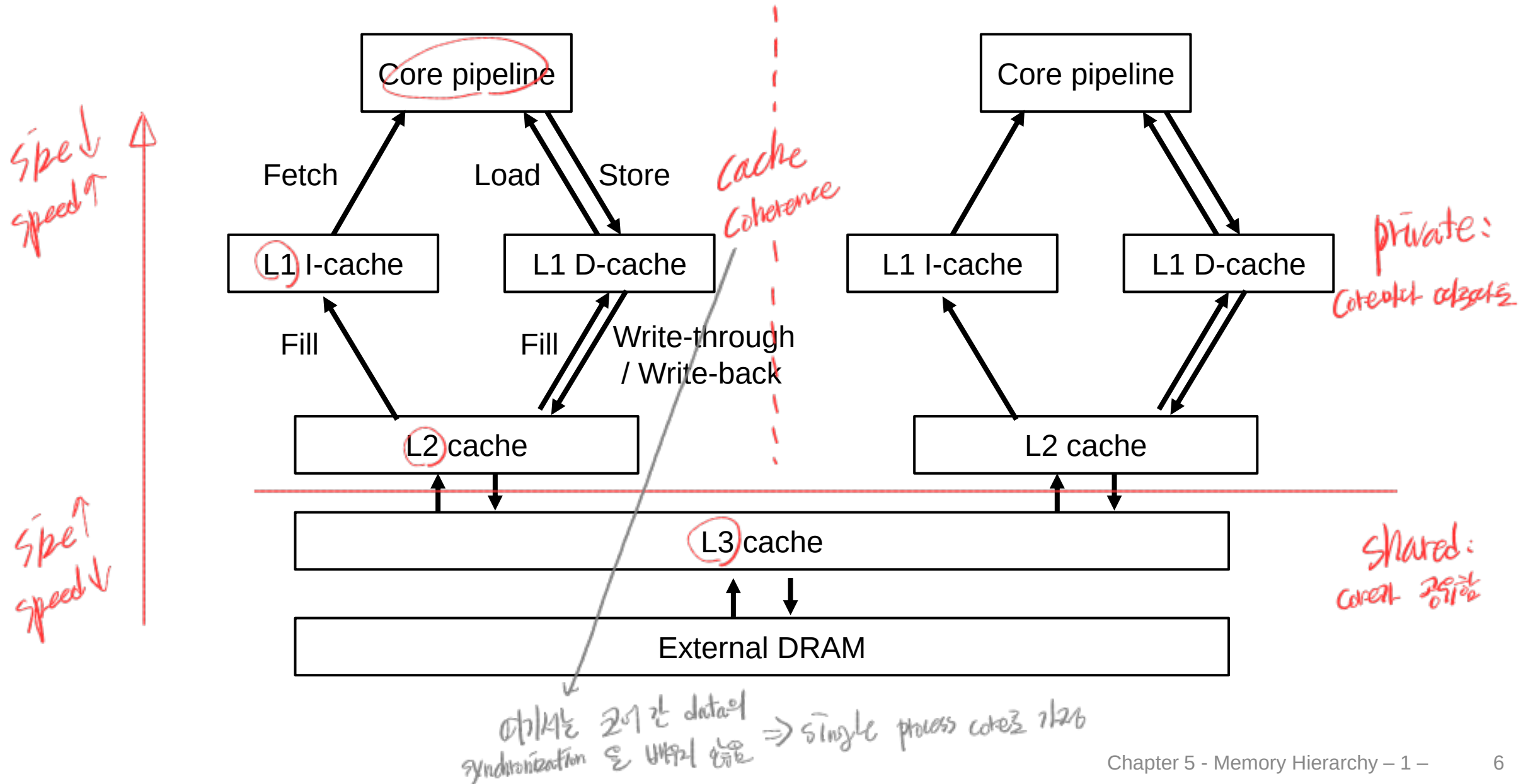
- ❖ Items near the recently accessed ones are likely to be accessed soon
- ❖ e.g., sequential instruction access, array data, stack frame



Typical Cache Hierarchy

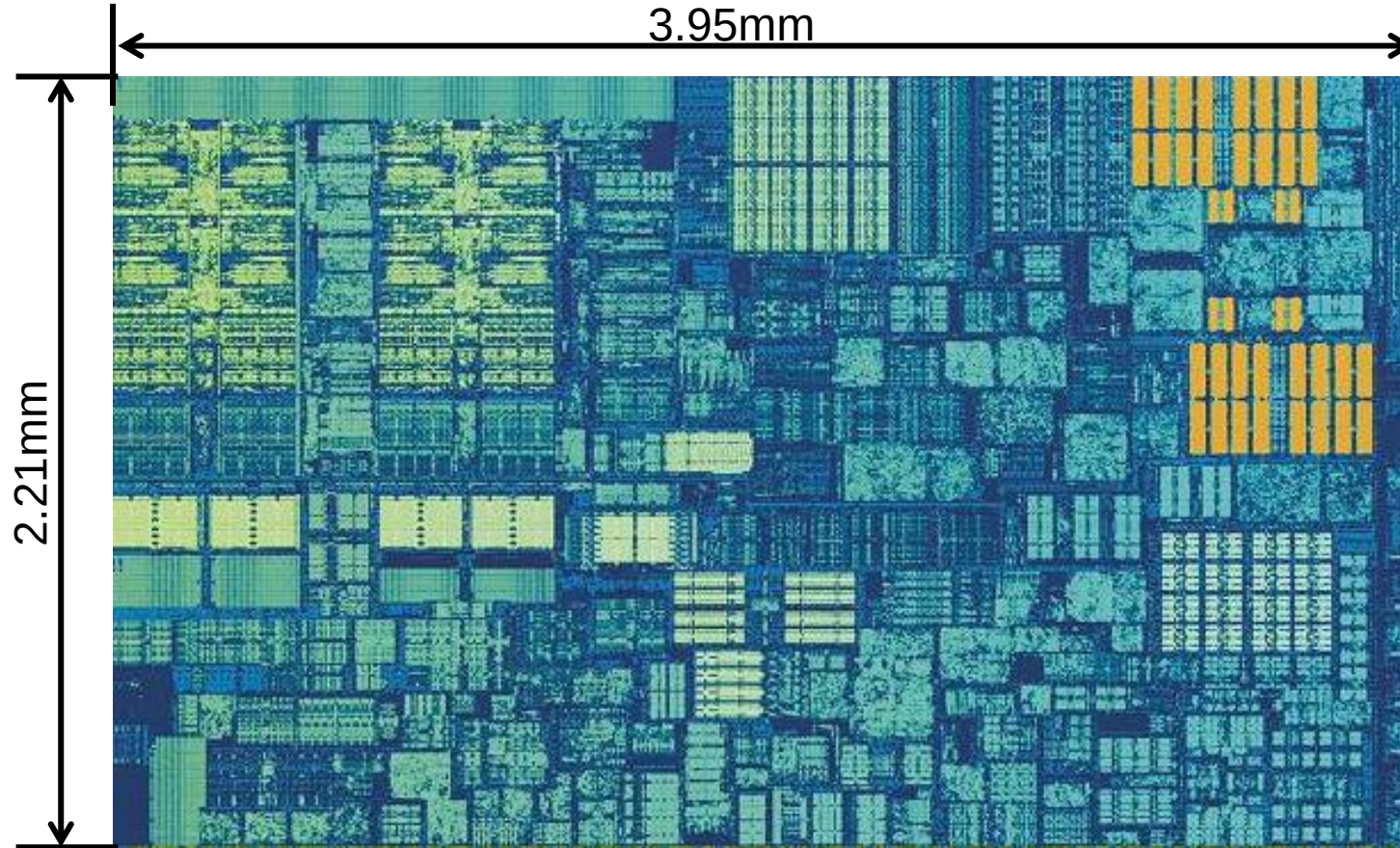


Typical Cache Hierarchy

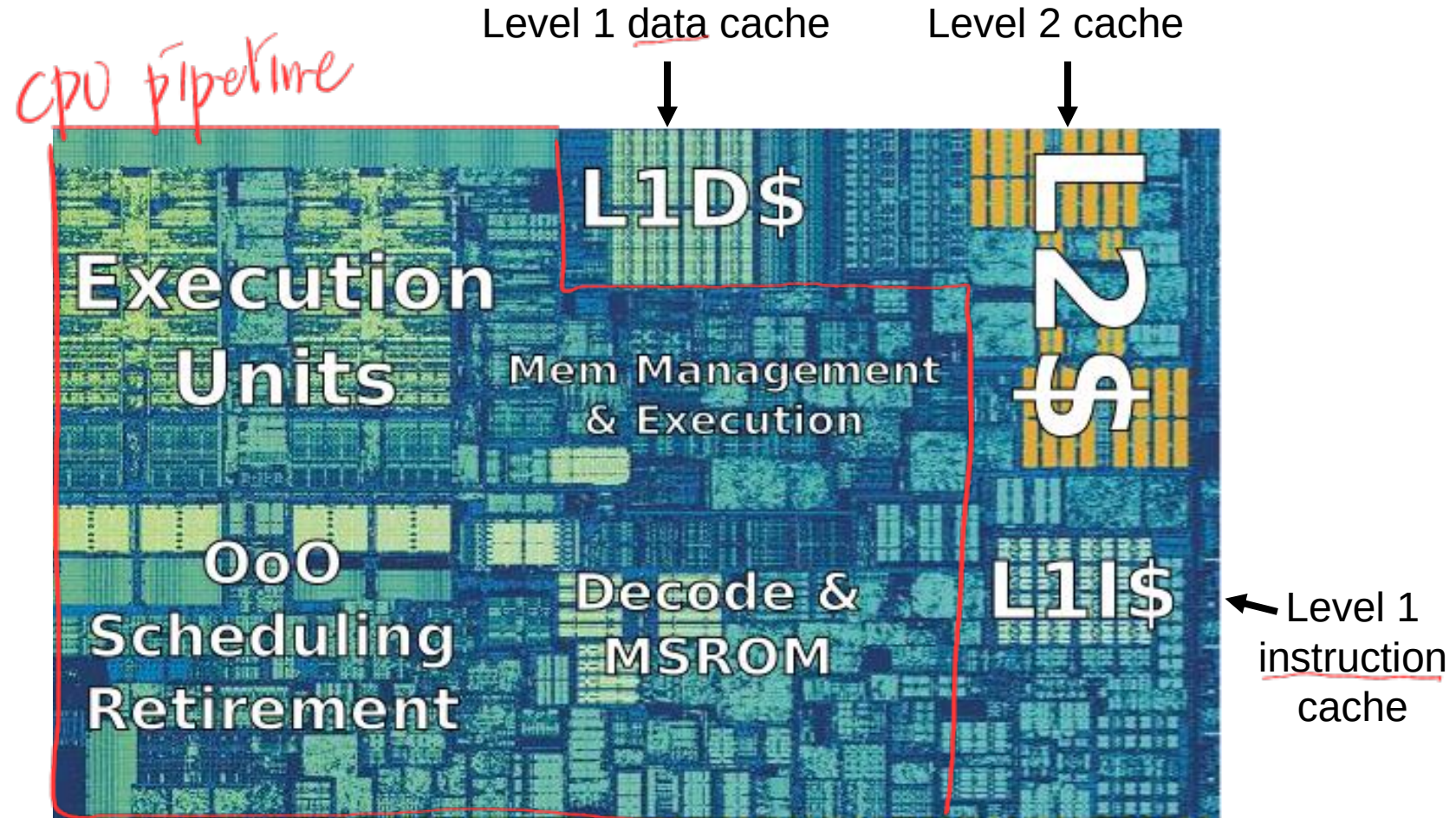


CPU Die Image

- Intel “Skylake” core

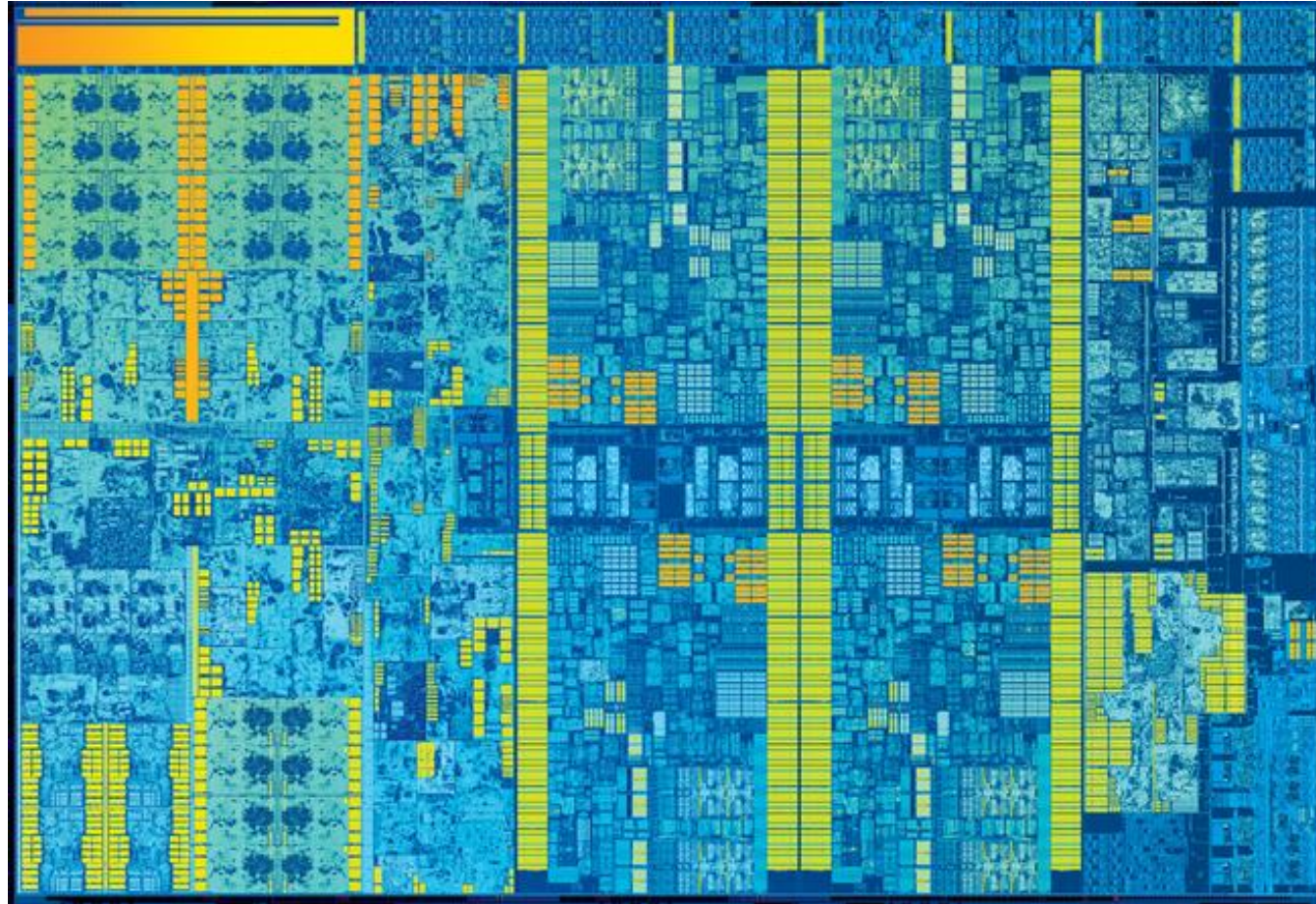


CPU Die Image



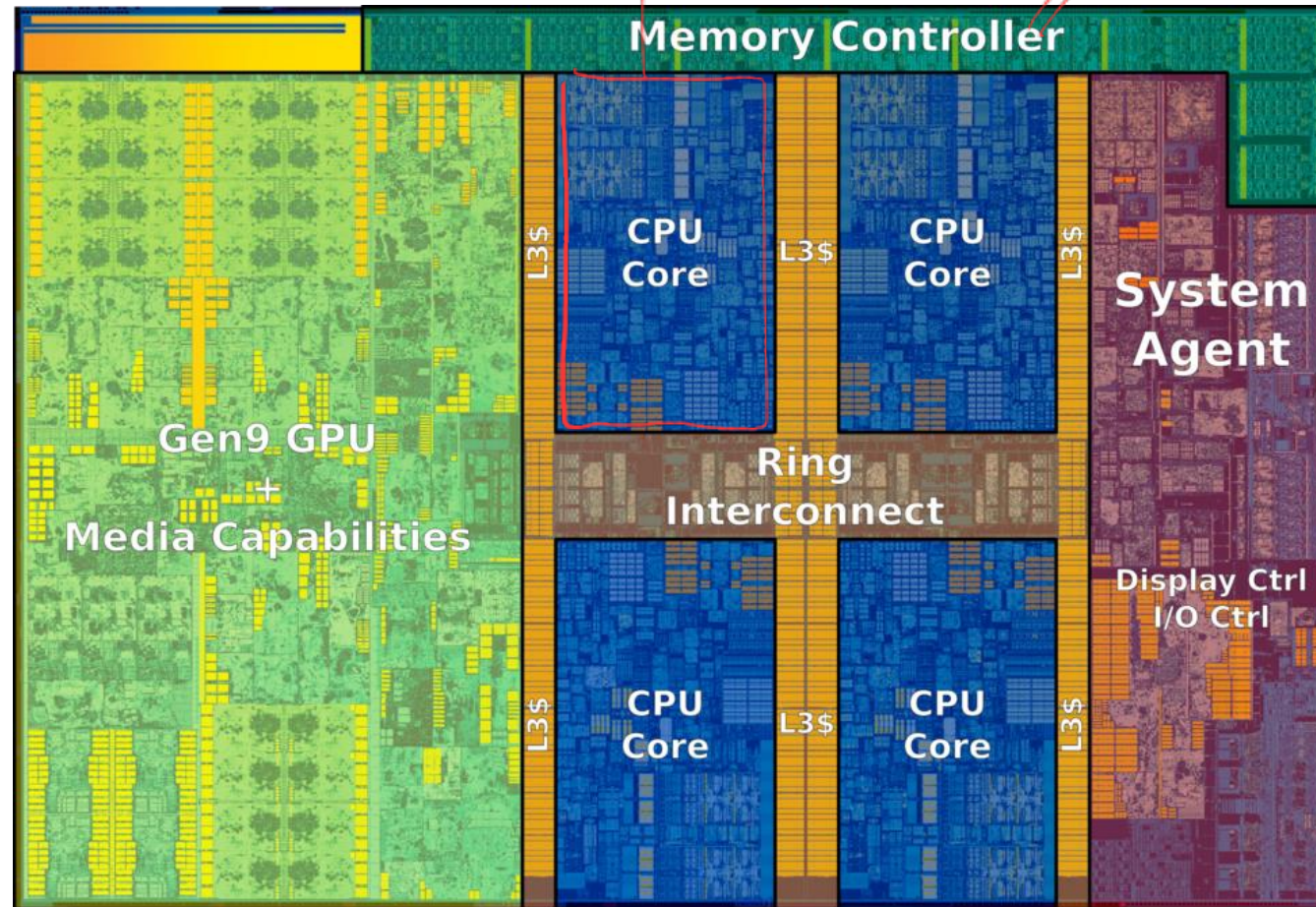
CPU Die Image

- Intel “Skylake” quad-core chip



CPU Die Image

- Intel “Skylake” quad-core CPU



Cache Memory

- Given accesses $X_1, \dots, X_{n-1}, X_n$
 - How do we know if data is present?
 - Where do we look?
specific cache memory 안 찾기

X_4
X_1
X_{n-2}
X_{n-1}
X_2
X_3

a. Before the reference to X_n

X_4
X_1
X_{n-2}
X_{n-1}
X_2
X_n
X_3

b. After the reference to X_n

Direct Mapped Cache

- Location determined by the address
- Direct mapped: only one choice
 - (Block address) *modulo* (# Blocks in cache)

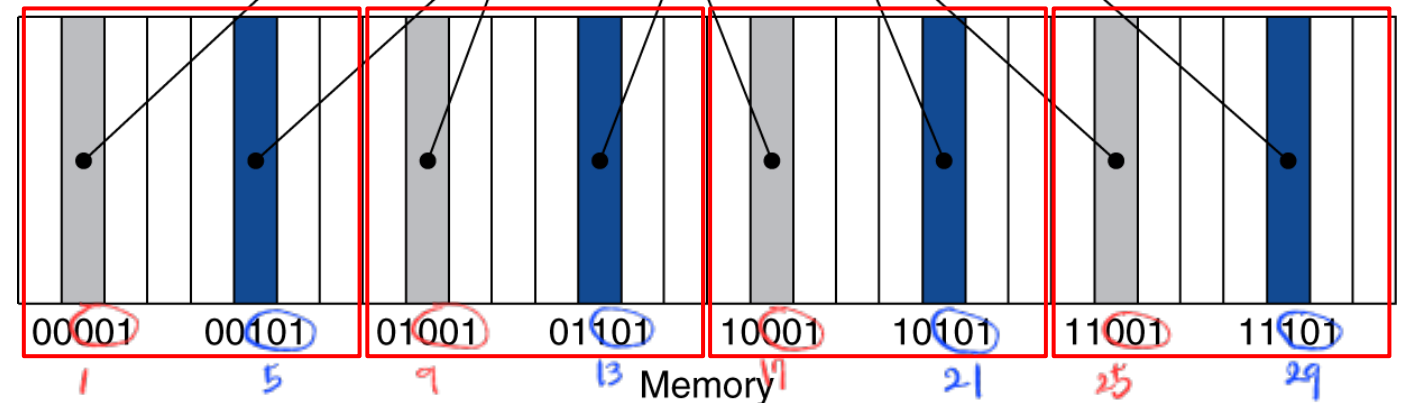
- # Blocks is a power of 2
- Use **low-order** address bits

$$X \bmod 2^y = \text{lower } y \text{ bits of } X$$

$$\bmod 2^5 \Rightarrow 010 \boxed{01010}$$

74 % 32 = 10

main memory



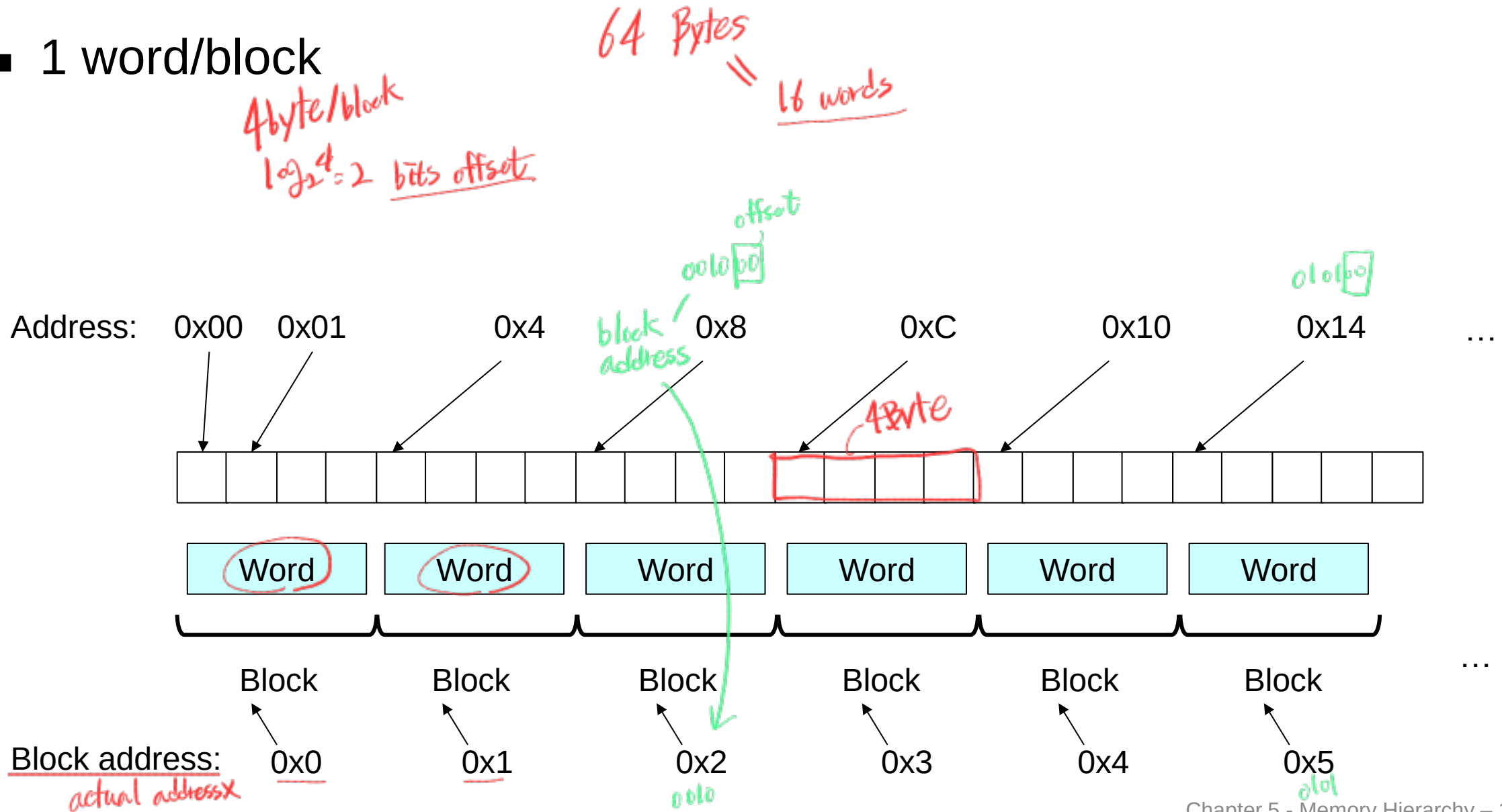
32 bit address

Tags and Valid Bits

- How do we know if a block is in the cache?
 - ❖ Store block address as well as the data
 - ❖ Only need the high-order bits
 - ❖ Called the tag ← tags 값
- What if there is no data in a location?
 - ❖ Valid bit: 1 = present, 0 = not present
 - ❖ Initially set to 0

Cache Example

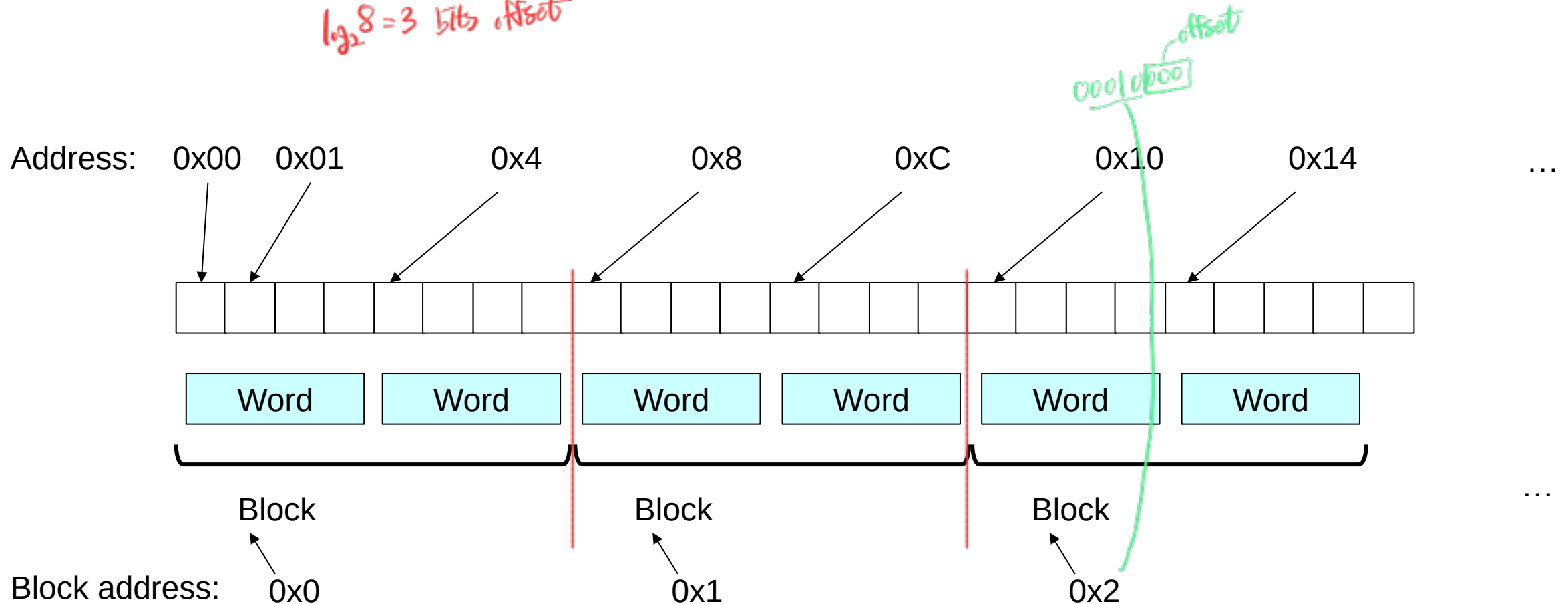
- 1 word/block



Cache Example

- Another example: 2 words/block

*8 byte/block
 $\log_2 8 = 3$ bits offset*

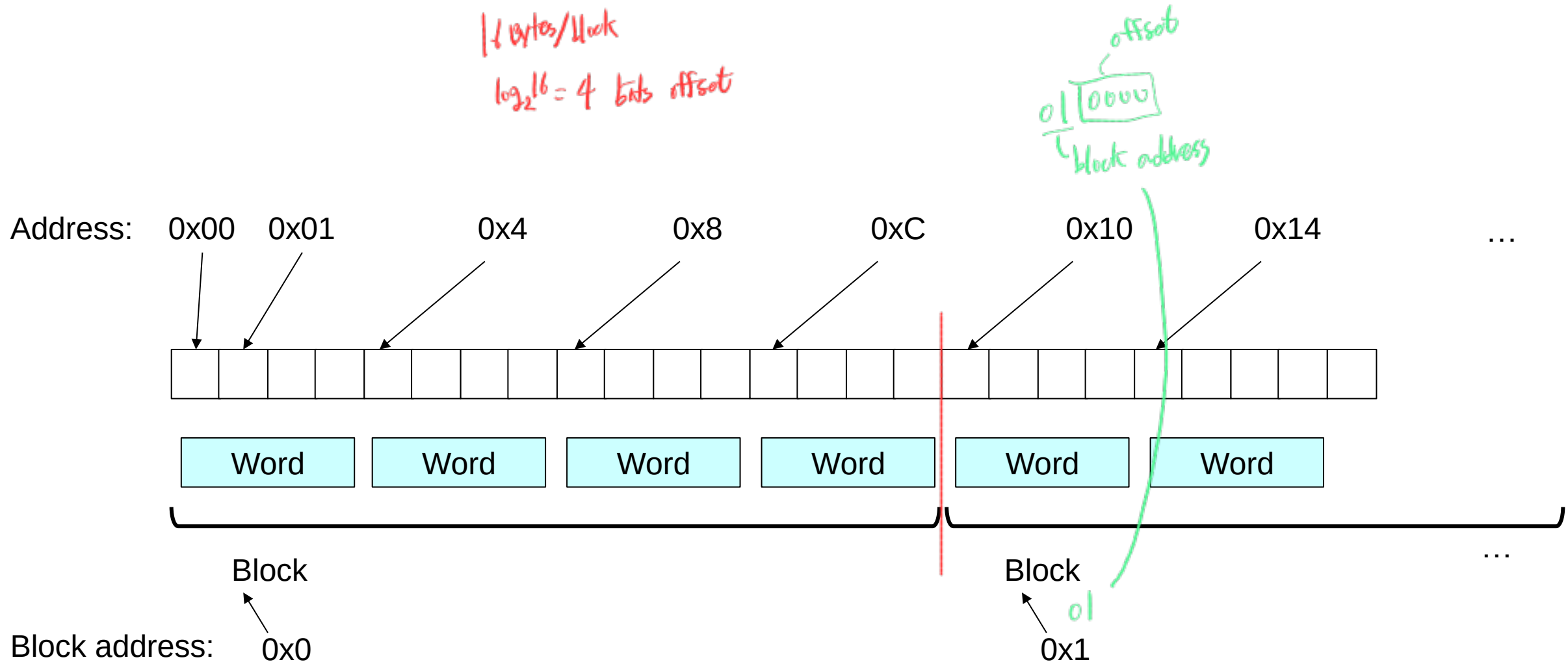


Cache Example

- Another example: 4 words/block

16 bytes/block

$\log_2 16 = 4$ bits offset



Address Decomposition

- Offset: Determined by the block size

- ❖ e.g., 1 word/block: 2 bits, 8 words/block: 5 bits

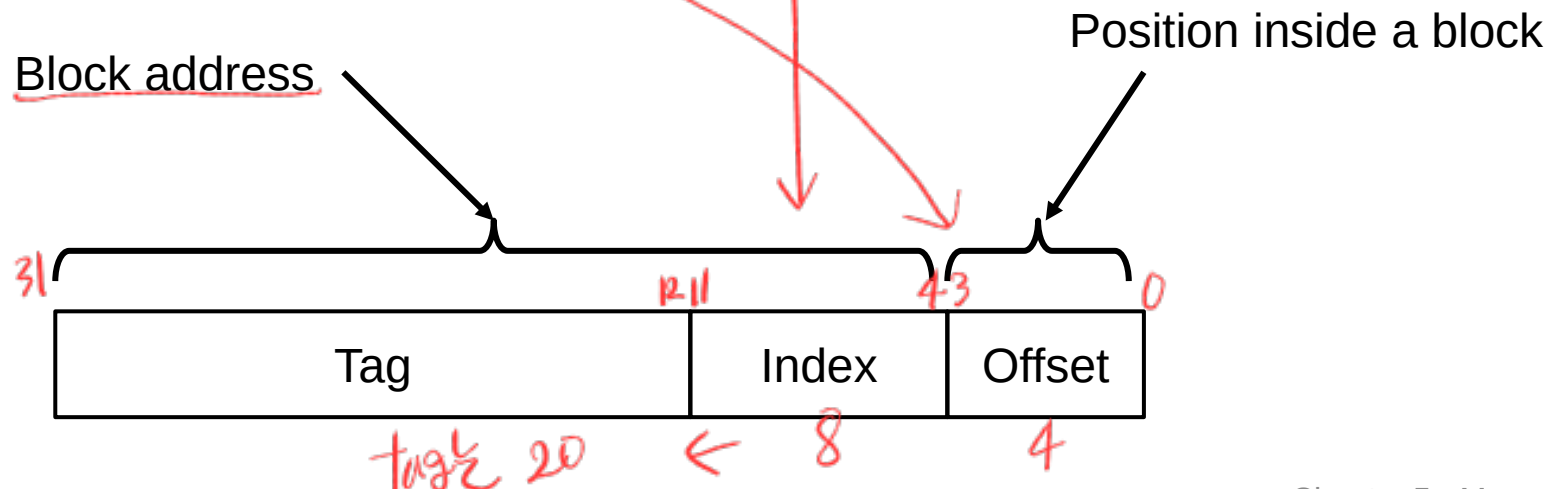
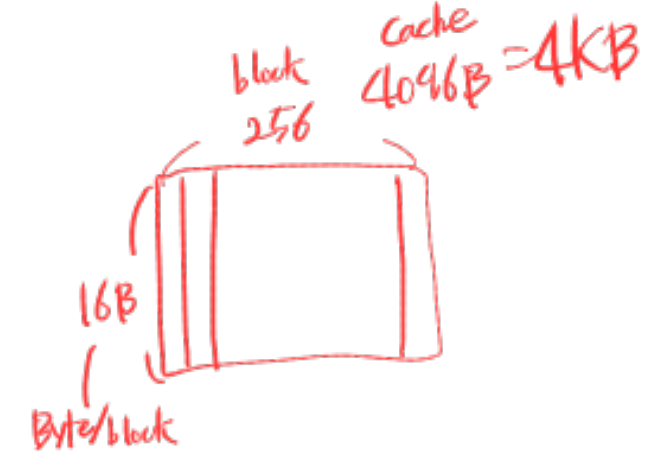
- Index: Determined by the number of blocks in cache

- ❖ e.g., 4KB cache, 4 words/block

1 block is 16 bytes. 4KB / 16B = 256 blocks in cache

$\log_2(256) = 8 \rightarrow$ 8-bit index

- Tag: The rest

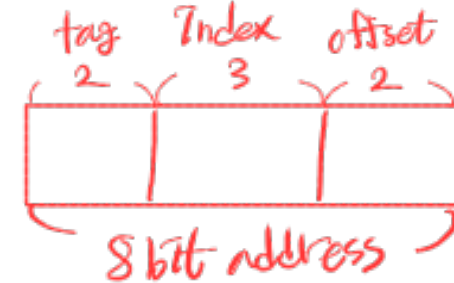


Cache Example

- Memory address: 8 bit
- 8-blocks, 1 word/block, direct mapped

$\log_2 8 = 3$ index

4B/block $\log_2 4 = 2$ 2 bit offset



- Initial state:

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Cache Example

Load 1

Address	Block address	Hit/miss	Cache block
0x58	010 110	Miss	110

0101 1000
tag index

Tag

Index

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	010	Mem[0x58]
111	N		

Cache fill (miss)

Cache Example

Load 2

Address	Block address	Hit/miss	Cache block
0x68	011 010	Miss	010

0110 1000

Tag *Index*

Index	V	Tag	Data
000	N		
001	N		
010	Y	011	Mem[0x68]
011	N		
100	N		
101	N		
110	Y	010	Mem[0x58]
111	N		

Cache fill

Cache Example

	Address	Block address	Hit/miss	Cache block
Load 3	0x58	010 110	Hit	110
Load 4	0x68	011 010	Hit	010

Index	V	Tag	Data
000	N		
001	N		
010	Y	011	Mem[0x68]
011	N		
100	N		
101	N		
110	Y	010	Mem[0x58]
111	N		

Cache Example

	Address	Block address	Hit/miss	Cache block
Load 5	0x40	010 000	Miss	000
Load 6	0x0C	000 011	Miss	011
Load 7	0x40	010 000	Hit	000

Index	V	Tag	Data
000	Y	010	Mem[0x40]
001	N		
010	Y	011	Mem[0x68]
011	Y	000	Mem[0x0C]
100	N		
101	N		
110	Y	010	Mem[0x58]
111	N		

Cache Example

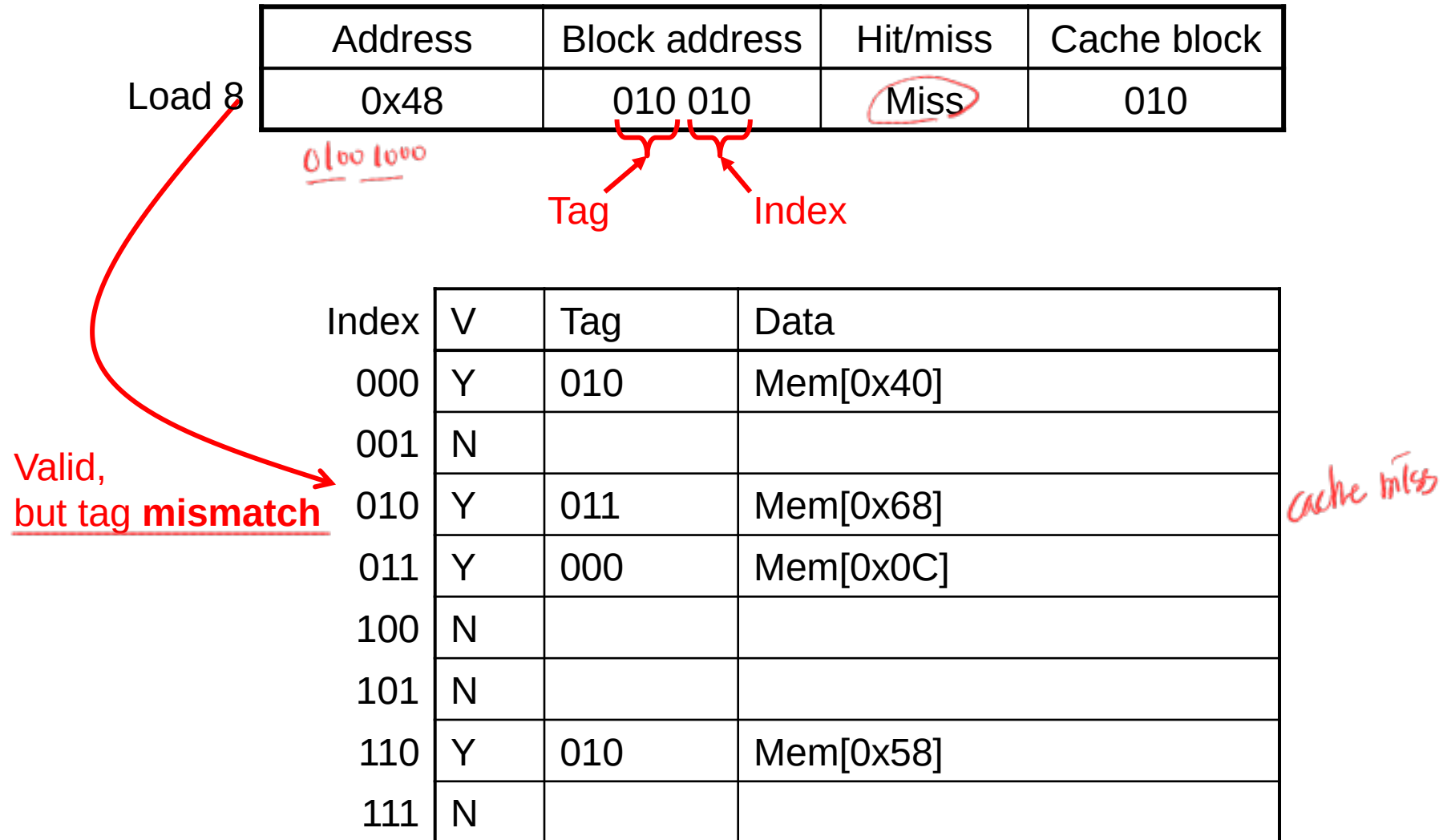


Diagram illustrating temporal locality:

Sequence of memory accesses: load_2 ($0x68$), load_4 ($0x68$), load_8 ($0x48$), load_2 ($0x68$). A red arrow points from the second load_2 to the first load_2 , indicating a repeat access.

Cache block
010

temporal locality

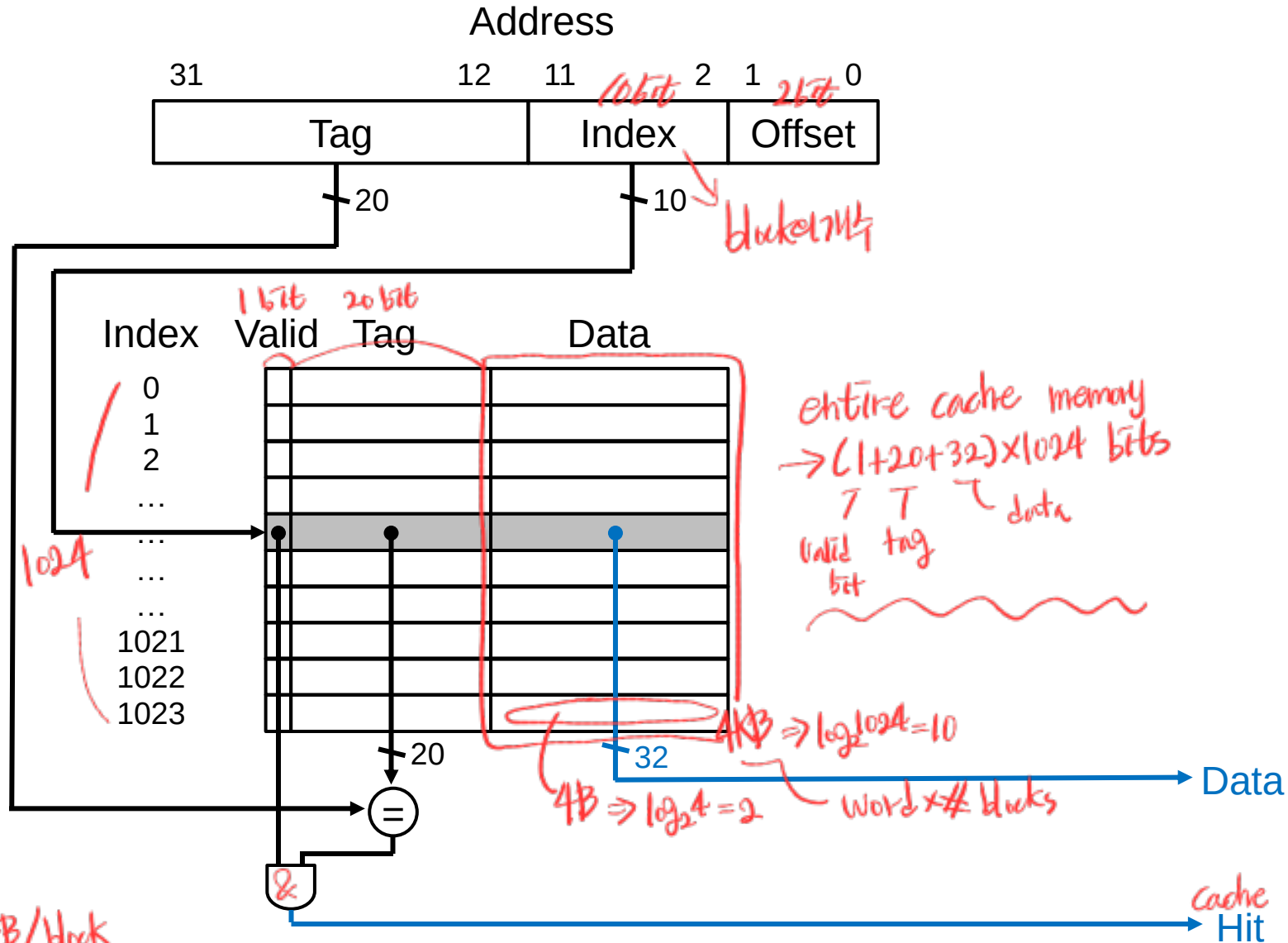
temporal locality

Replace

new cache fill
overwrite
old data

Kick out old
bring new

Address Subdivision (1)



Index를 가리키

Tag² 확인

Valid 유효한



hēt

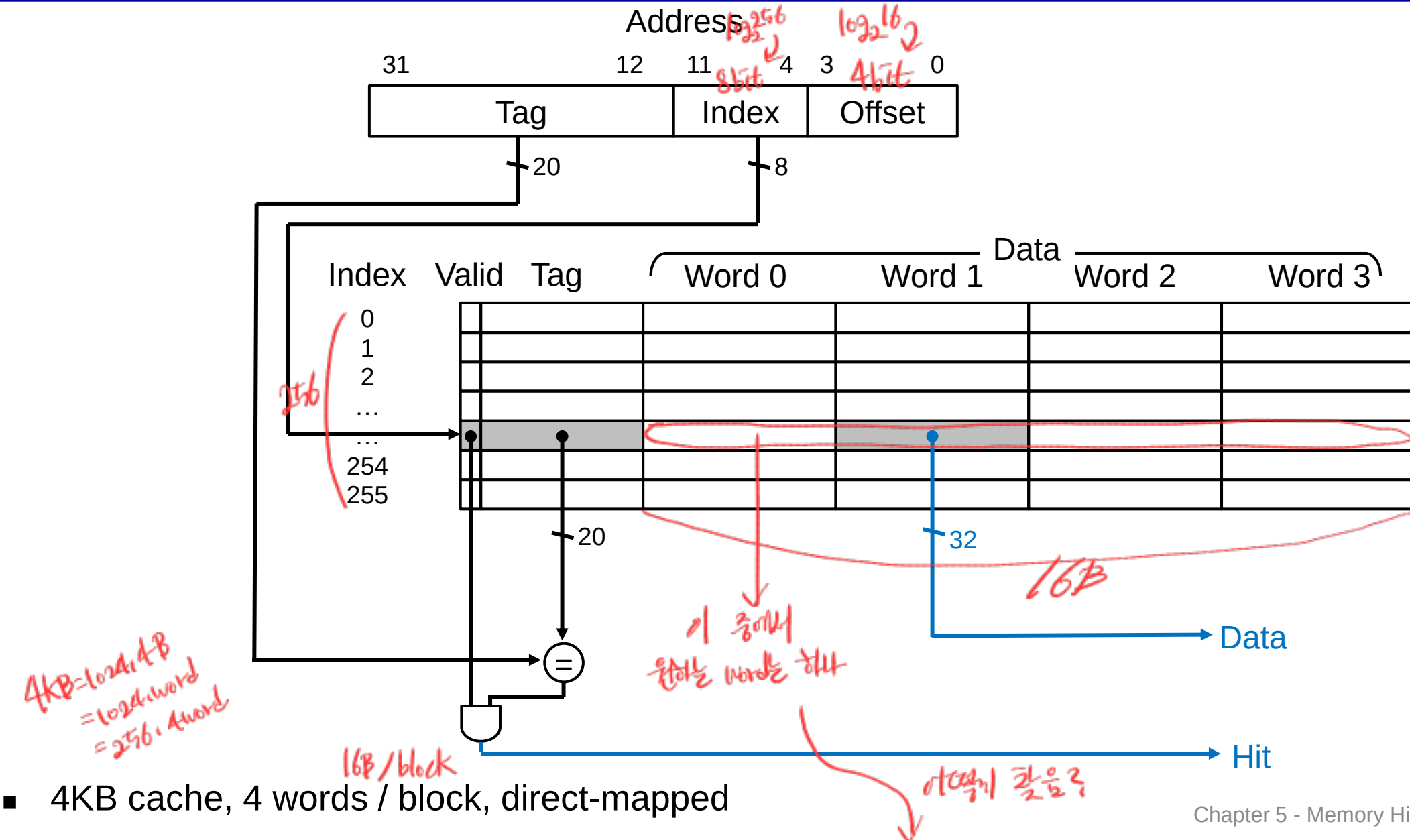
(하트세이브즈
동네에 관해)

$$4KB = 1024 \times 4B = 1024 \text{ word}$$

32bit, 4B/block

- 4KB cache, 1 word / block, direct-mapped

Address Subdivision (2)

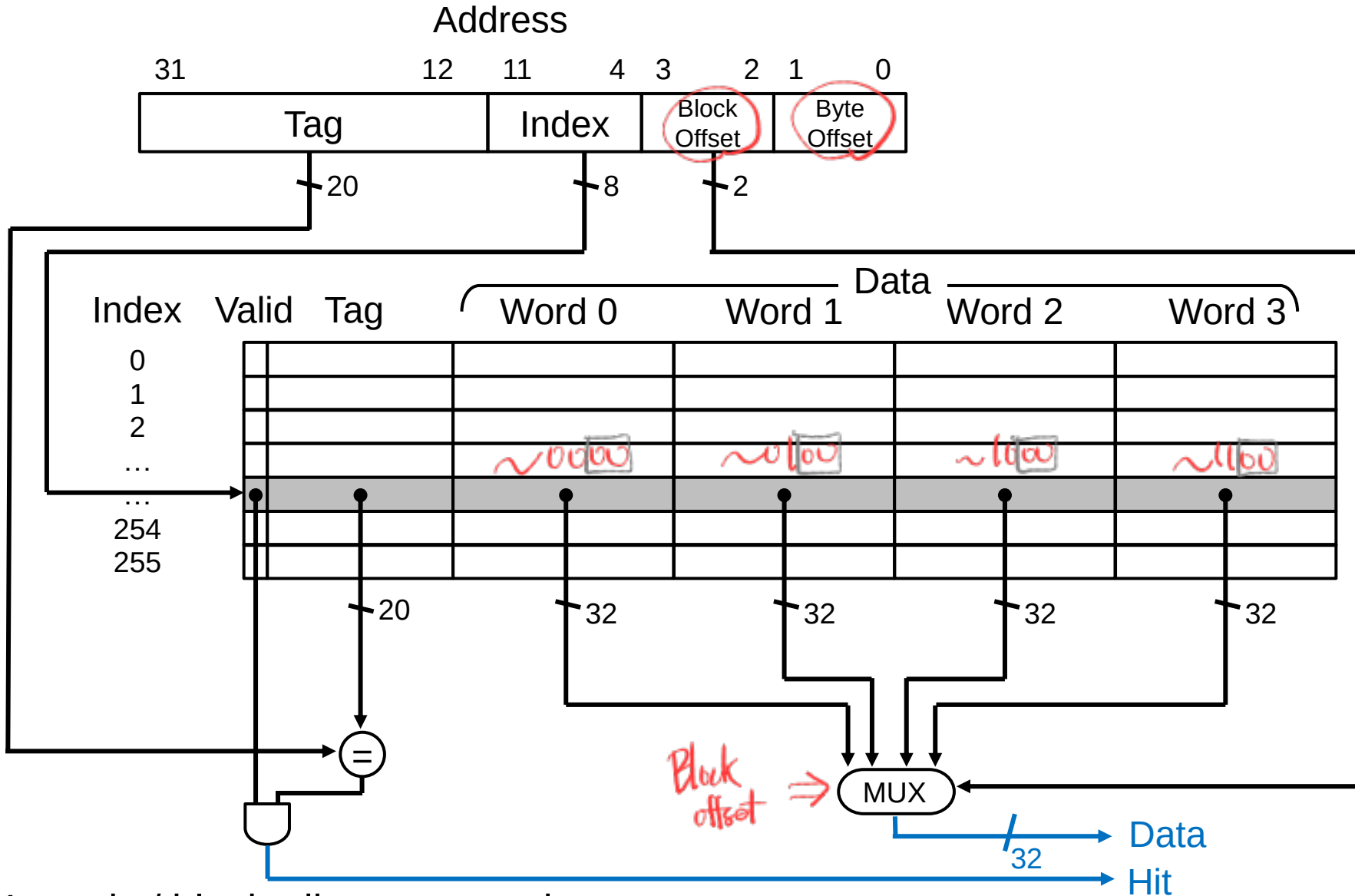


- 4KB cache, 4 words / block, direct-mapped

Address Subdivision (3)

offset의 위치

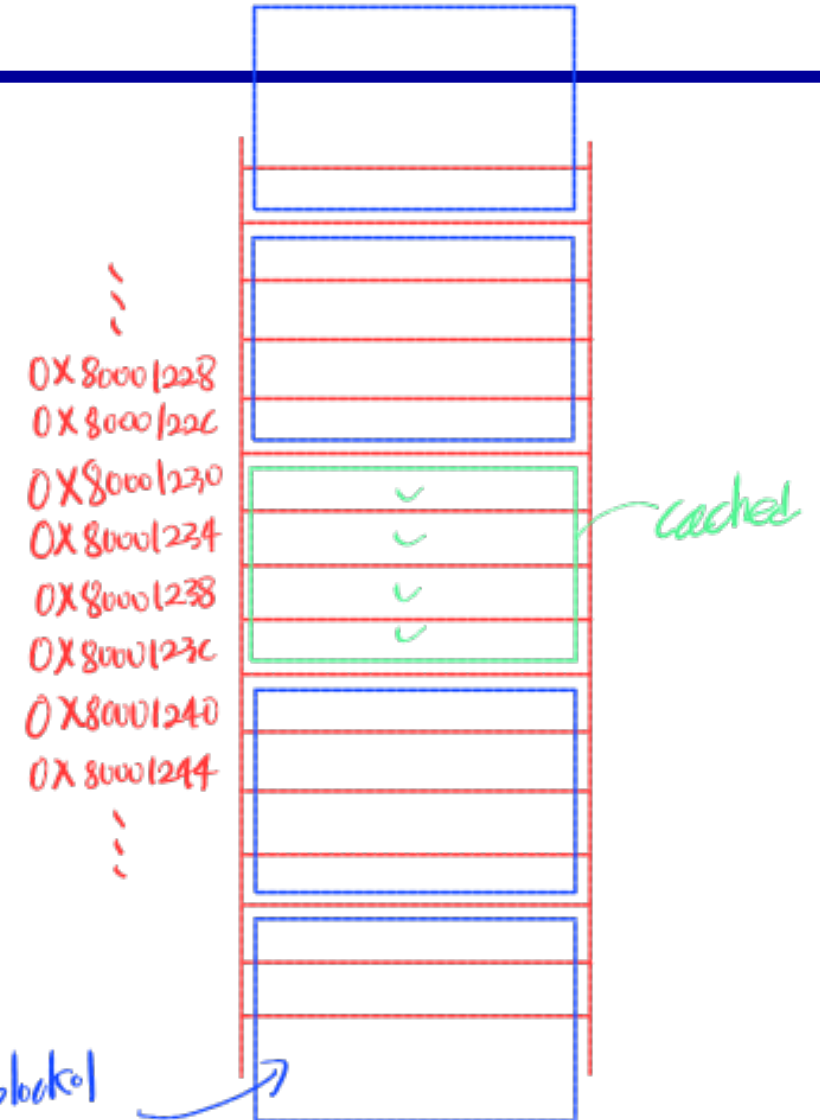
1W-4B(word)



- 4KB cache, 4 words / block, direct-mapped

Block Size Considerations

- Larger blocks should reduce miss rate
 - ❖ Spatial locality
- Example: 32bit address, 4 words / block
 - ❖ If there is a miss on address 0x80001234...
 - ❖ ... cache is filled with the following 4 words
 - 0x80001230 ← 0x80001234X
 - 0x80001234
 - 0x80001238
 - 0x8000123C



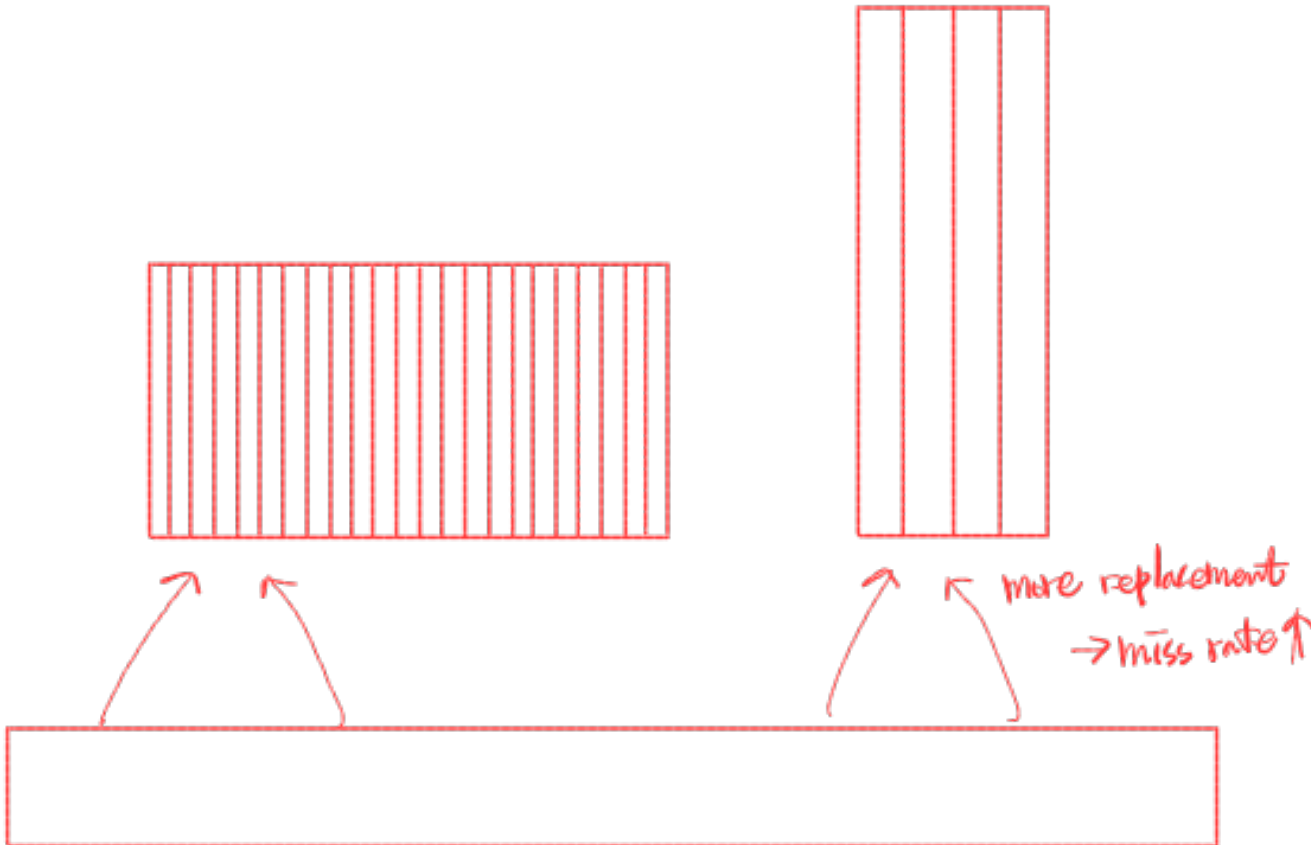
Block Size Considerations

larger block $\Rightarrow$ ~~block~~ $\downarrow$
smaller block $\Rightarrow$ ~~block~~ $\uparrow$

■ ...but in a fixed-sized cache,

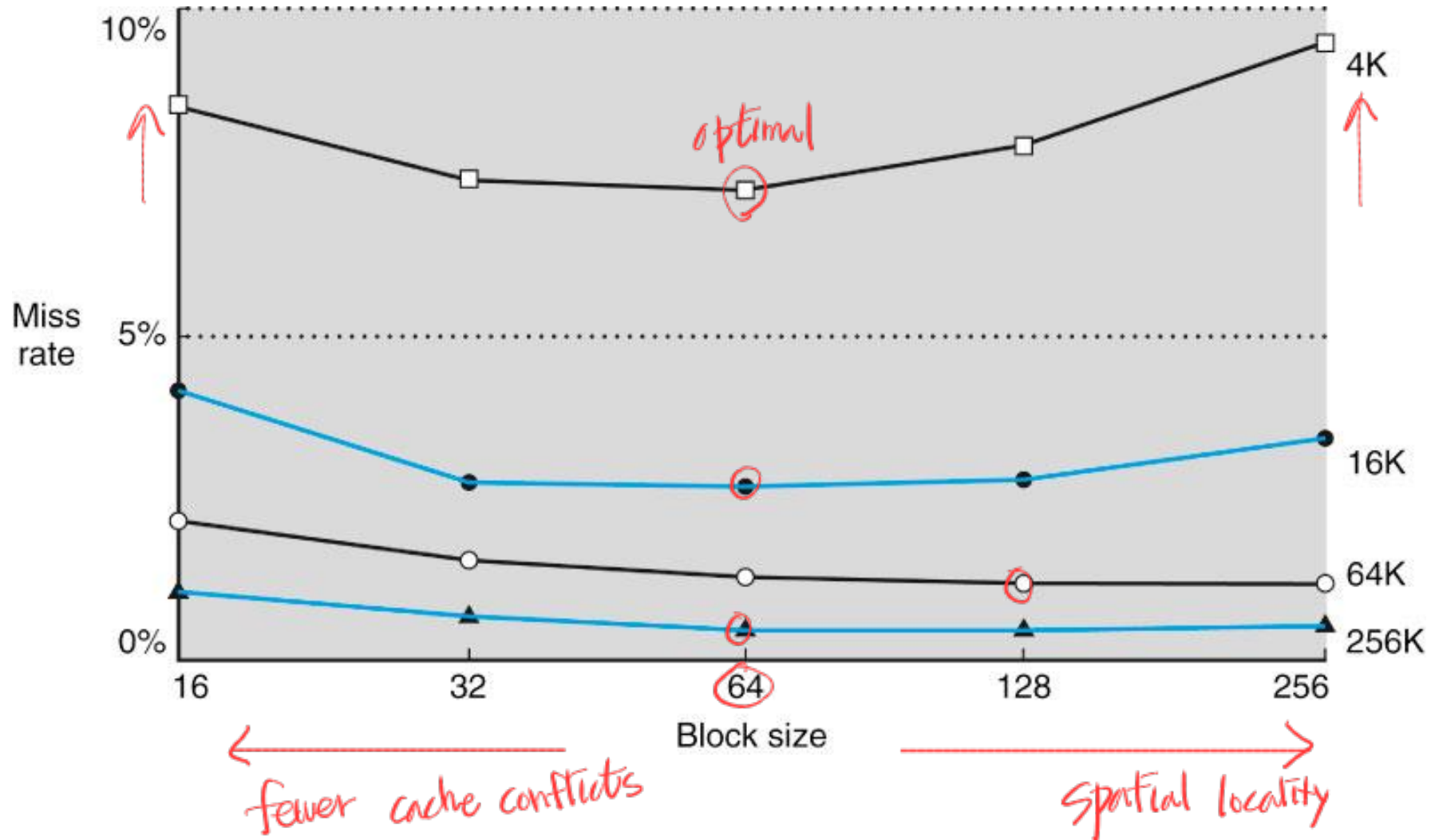
❖ Larger blocks $\Rightarrow$ smaller number of cache lines $\Rightarrow$ more competition

$\hookrightarrow$ more replacement (pollution) $\Rightarrow$ **higher miss rate**



Block Size Trade-off

simulate



Cache Misses

- On cache hit, CPU proceeds normally
- On cache miss
 - ❖ Stall the CPU pipeline *stop cpu pipelining*
 - ❖ Fetch block from next level of hierarchy *L1 miss? → L2, L3, ...*
 - ❖ Instruction cache miss
 - Restart instruction fetch
 - ❖ Data cache miss
 - Complete data access

Handling Write (Store) Operations

■ Store hit:

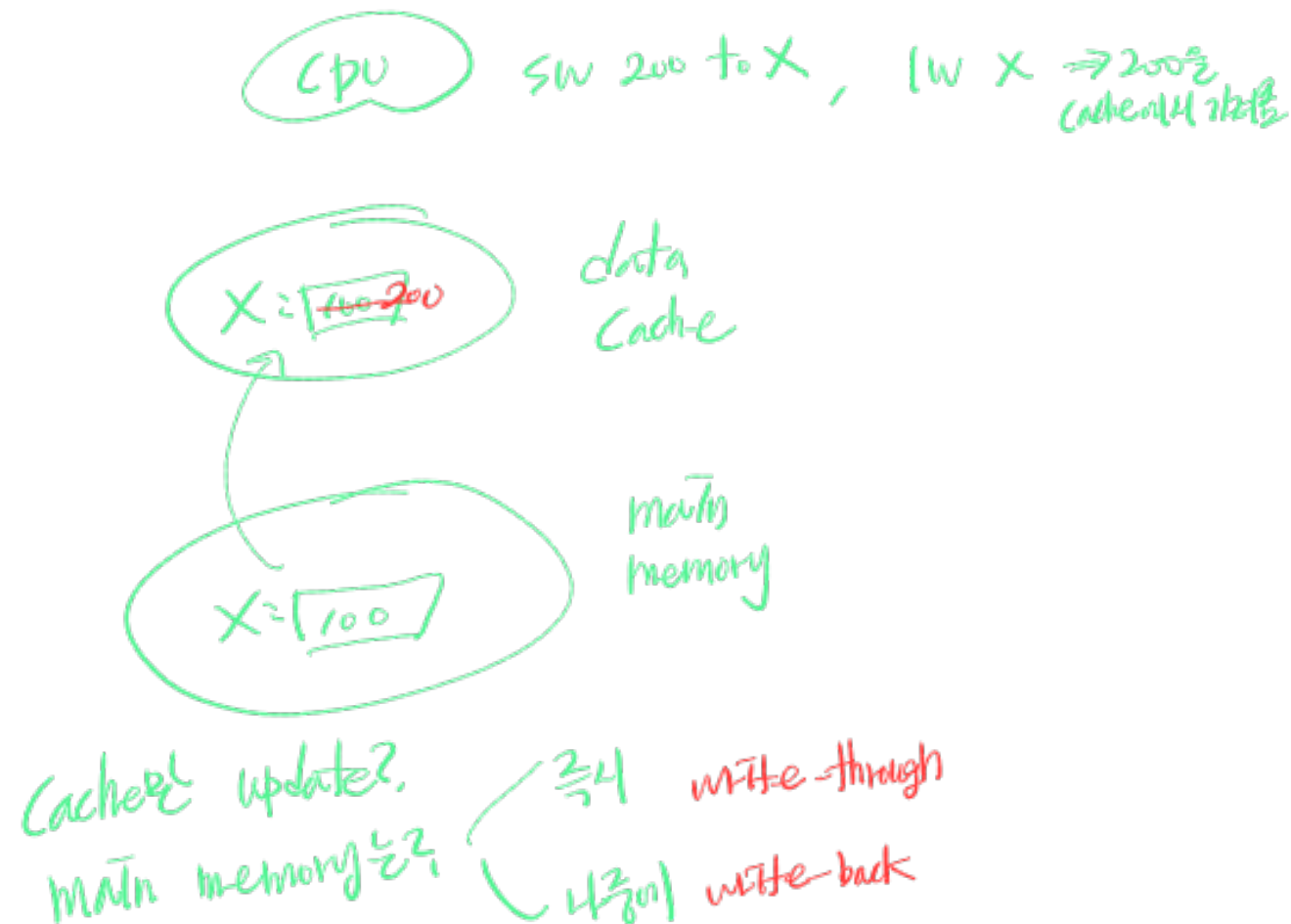
- ❖ Update cache?
- ❖ Update memory?

■ Store miss:

- ❖ Fetch the line to the cache?

→ X의 store를 위해 캐시에서,
cache에 없는 상황

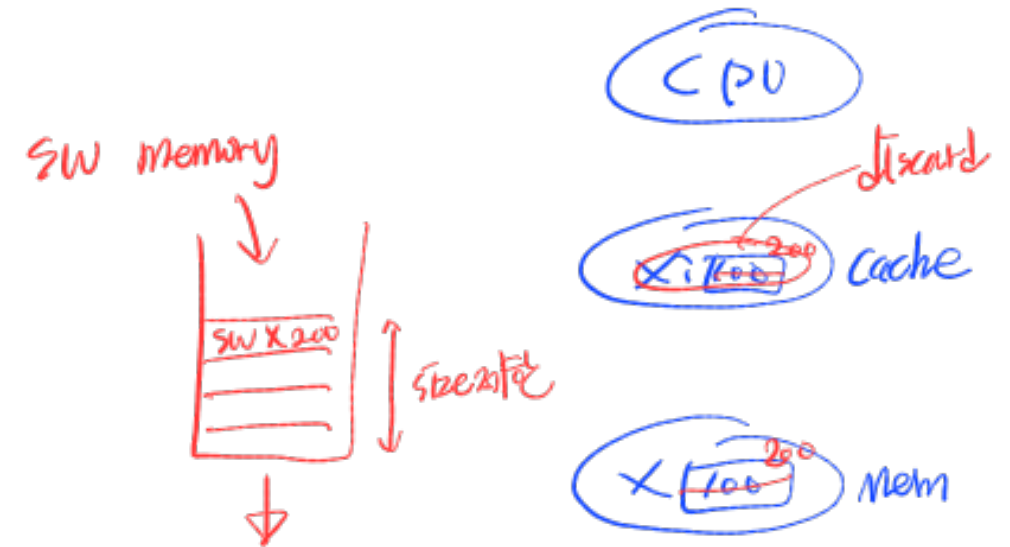
- ① data cache에서,
main memory로 가기
- ② data cache에 load,
perform store operation



Write-Through

- On write, also update memory immediately
cache & memory update, same address엔 같지만 same data
- ...but writes take a long time
 - ❖ e.g., if base CPI = 1, 10% of instructions are stores, write to memory takes 100 cycles
 - Effective CPI = $1 \times 100\% + 100 \times 10\% = 11$

- Solution: write buffer *FIFO*
 - ❖ Holds data waiting to be written to memory
 - ❖ CPU continues immediately
 - Stall on write if only the write buffer is full
store buffer
만약이면 계속 stall

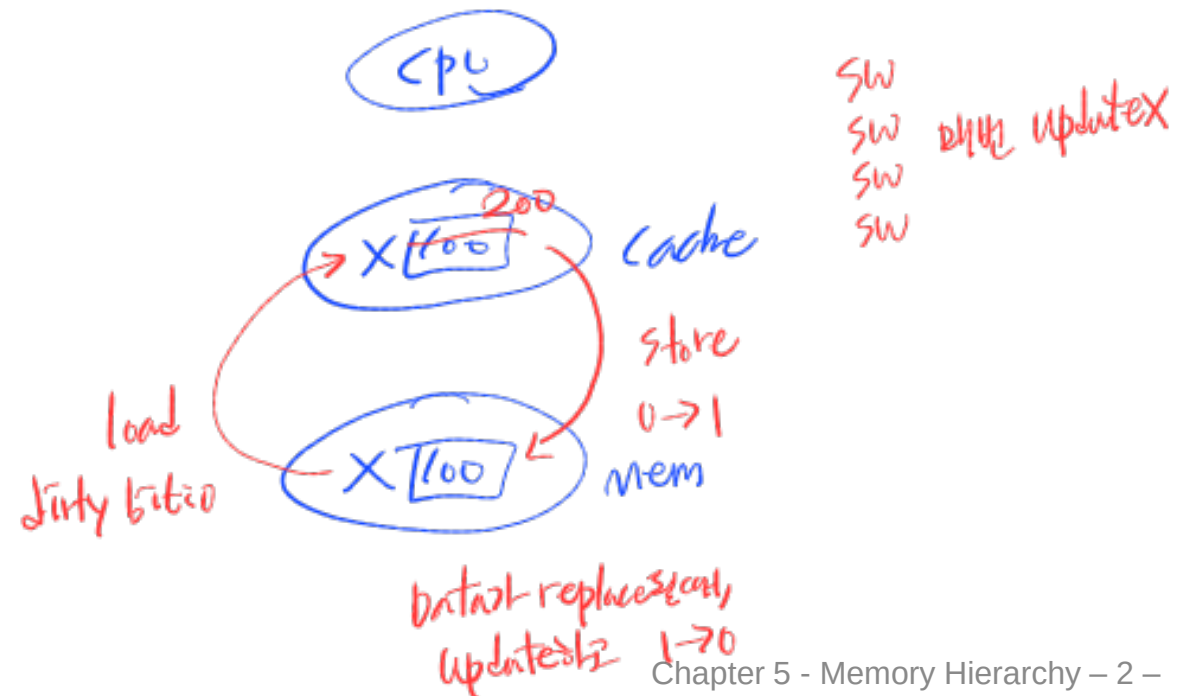


Write-Back

- On data-write hit, just update the block in cache
 - ❖ Keep track of whether each block is **dirty** (D bit)

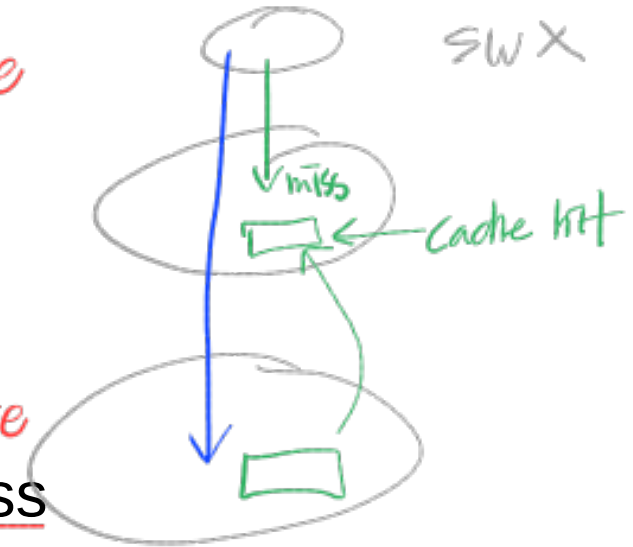


- When a dirty block is replaced
 - ❖ Write it back to memory
 - ❖ This also uses the write buffer



Handling Store Miss

- **No write allocate** ① *cache는 건들지 않고 memory만 update*
 - ❖ Just go to the memory and update memory only
 - ❖ Cache is unchanged.
- **Write allocate** ② *cache가 없다면 cache에 load하고 update*
 - ❖ Fetch the line from memory as if we have a load miss
 - ❖ ... and then do the same thing as a store hit.



Cache Write Scheme Summary - 1

- Write-through, write allocate
 - ❖ Write hit: update cache, update memory
 - ❖ Write miss: fetch the block, update cache, update memory
cache에 블록 할당

- Write-through, no write allocate
 - ❖ Write hit: update cache, update memory
 - ❖ Write miss: update memory *cache에 할당 안함*

	W T	W B
WA	hit/miss	hit/miss
NWA	hit/miss	hit/miss

4/21

Cache Write Scheme Summary - 2

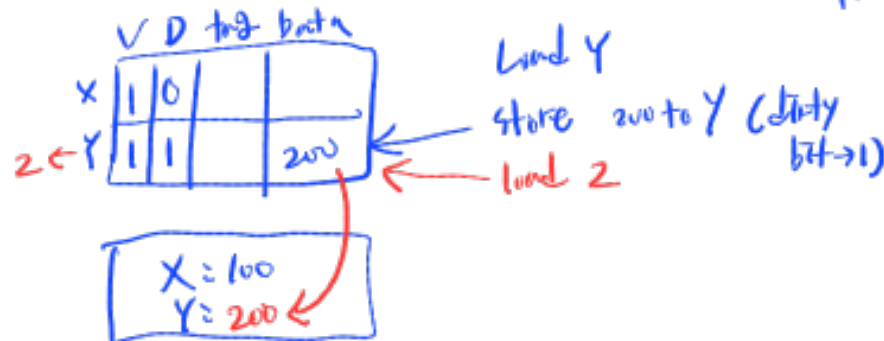
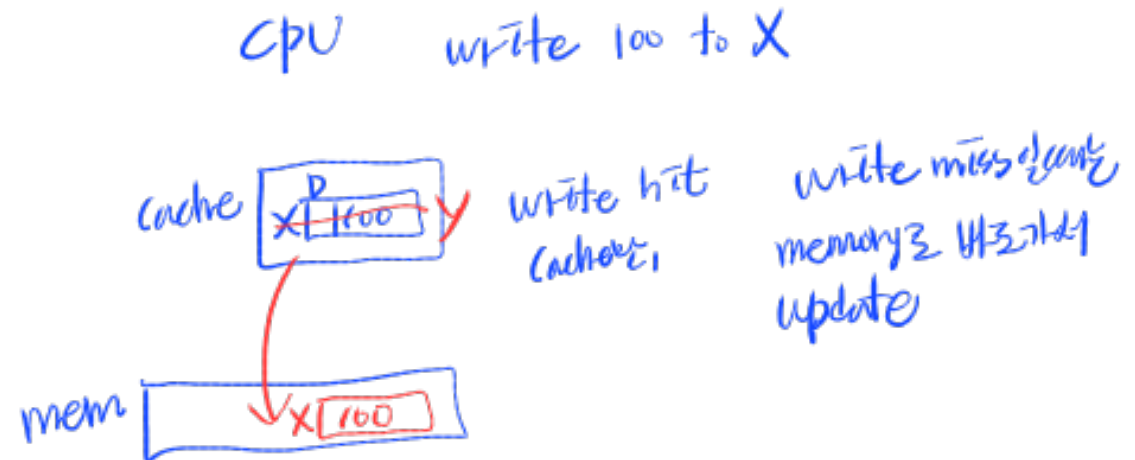
■ Write-back, write allocate

- ❖ Write hit: update cache (set dirty bit)
- ❖ Write miss: fetch the block, update cache (set dirty bit)
- ❖ Dirty line replace: update memory

dirty bit
→ main memory
Cache block
replace & can update

■ Write-back, no write allocate

- ❖ Write hit: update cache (set dirty bit)
- ❖ Write miss: update memory *Cache 2 miss*
- ❖ Dirty line replace: update memory



Measuring Cache Performance

- Components of CPU time
 - ❖ Program execution cycles
 - ① how it slows down CPU behavior
 - ② how long it will take to access memory location
 - ⊕ ➤ Instruction cycles without any extra stalls from memory ~~stalls~~ (hazard)
 - ❖ Memory stall cycles
 - Cache miss penalties

- With simplifying assumptions:

$$\begin{aligned} & \text{Memory stall cycles} \\ &= \frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty} \\ &= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty} \end{aligned}$$

constant for this lecture only

Cache Performance Example

■ Given

- ❖ I-cache miss rate = 2%
- ❖ D-cache miss rate = 4%
- ❖ Miss penalty = 100 cycles for a load/store miss
- ❖ Base CPI (ideal cache: all hits) = 2
- ❖ Load & stores are 36% of instructions

98% addition cycle X (cache hit)
96%

Base CPI에
hit time이
포함되어
있음

■ Miss cycles per instruction

- ❖ I-cache: $0.02 \times 100 = 2$
- ❖ D-cache: $0.36 \times 0.04 \times 100 = 1.44$

all instruction
load/store

- Actual CPI = $2 + 2 + 1.44 = 5.44$
 - ❖ Ideal CPU is $5.44 / 2 = 2.72\times$ faster

$$I = 2\% \times 100 = 2$$
$$D = 4\% \times 36\% \times 100 = 1.44$$

$$I + D = 3.44$$

Base = 2

$$5.44 / 2 \rightarrow 2.72\times \text{ faster}$$

Average Access Time

- Hit time is also important for performance

- Average memory access time (AMAT)

- ❖ $\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$

- Example

- ❖ CPU with 2ns clock, hit time = 1 cycle, miss penalty = 20 cycles, I-cache miss rate = 6%

- ❖ $\text{AMAT} = \underbrace{2\text{ns} \times 100\%}_{\text{hit time}} + \underbrace{40\text{ns} \times 6\%}_{\text{miss}} = 2\text{ns} + 2.4\text{ns} = 4.4\text{ns}$

Base CPL

$$\begin{aligned}\text{AMAT} &= 100\% \cdot 2\text{ns} + 6\% \cdot 40\text{ns} \\ &= 2\text{ns} + 2.4\text{ns} = 4.4\text{ns}\end{aligned}$$

1ns clock $\rightarrow 1\text{GHz}$

2ns " $\Rightarrow 0.5\text{GHz} = 500\text{MHz}$

40ns